YASC (Yet Another Snake Clone)

Realtime Interactive Systems Wintersemester 2020/2021

Felix Riehm
Matrikelnummer: 575067

22. Januar 2021

Inhaltsverzeichnis

Einleitung 3
Projektstruktur 4
Projektinstallation 7
Netzwerk und Architektur 8
Server 8
Client 9
Synchronisation 9
Limitationen 10
Grafik 12
Shader und Renderer 12
Textrenderer 13
LWJGL 13
Welt 14
Hauptschleife 14
Sichtfeld 15
Externe Ressourcen 17

Einleitung

YASC ist ein in Java geschriebenes Multiplayer-Spiel mit Battle Royale Ansatz. Es handelt sich dabei grundsätzlich um Snake, welches mit einer zufallsgenerierten dynamischen Welt, MVC-Architektur, Client-Server-Architektur und einer AI erweitert wurde. Für die Darstellung wird OpenGL verwendet. Als Datenstruktur für die Kollisionserkennung wird ein Quadtree verwenden und für die Zufallsgenerierung der Diamond Square Algorithmus. Das Ziel des Spiels ist es als allerletzter Spieler übrig zu bleiben und sich mit dem Einsammeln von Futtern zu vergrößern. Das Projekt wurde zusammen mit Jördis Liermann erstellt, in deren Dokumentation die AI beschrieben wird. In Abbildung 1 ist das Spiel in Aktion zu sehen.

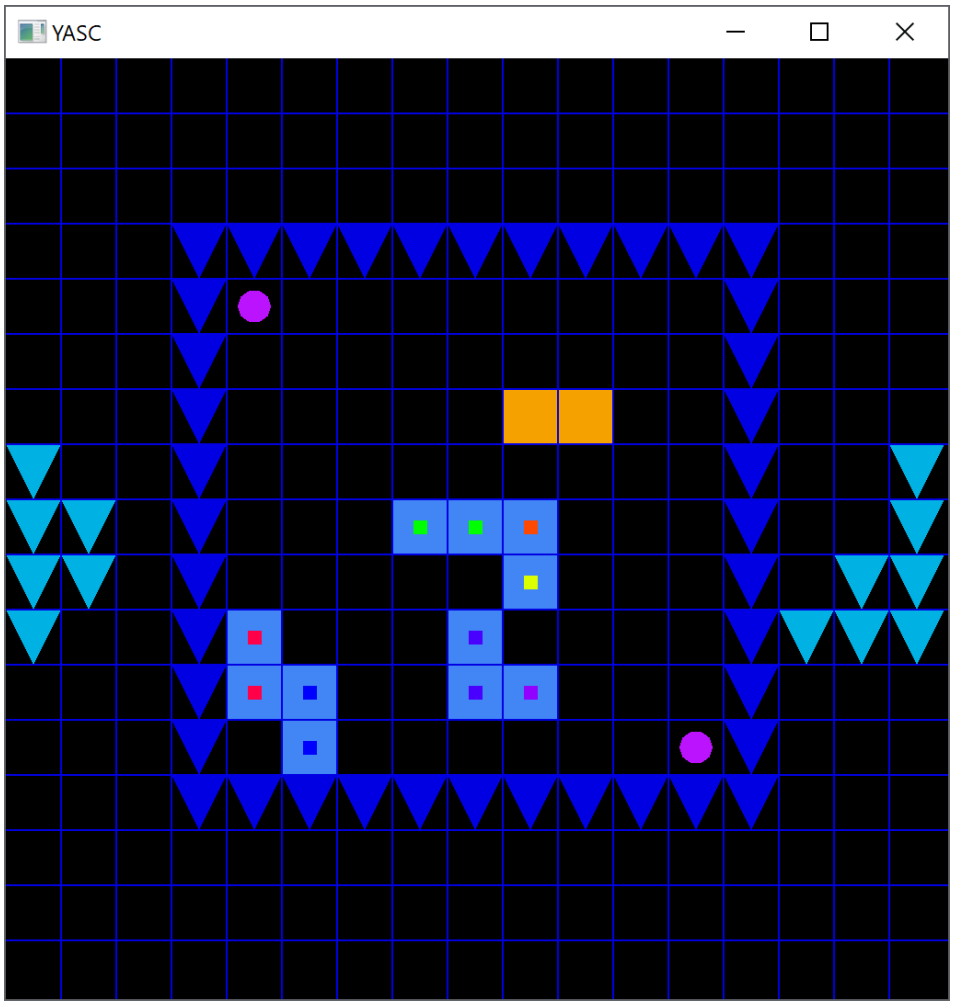

Abbildung 1: Screenshot von YASC. Die Mauer hat sich im Spielverlauf verkleinert. Orange Quadrate: Eigener Spieler. Lila Kreise: Futter. Hellblaue Quadrate: Gegner (Farbe des inneren Quadrats identifiziert Spieler). Dunkelblaue Dreiecke: Mauer die kleiner wird. Hellblaue Dreiecke: Ruinen (weitere Hindernisse).

Projektstruktur

Die Projektstruktur ergibt sich wie folgt:

Client

Das Client Package enthält Klassen und Packages, die nur für den Client relevant sind:

- **Controller**
  - **Game** (Game.java)
    - Anlegen der Datenstruktur, die vom Server kommt und von der View gerendert werden soll.
    - Handelt es sich um einen AI Client: Aufrufen des AI Controllers
  - **Network**
    - Thread zum Lesen und bearbeiten der Nachrichten (MsgReader.java)
    - Herstellen der Verbindung zum Server, Erstellen des MsgReaders und Senden von Nachrichten (NetworkManager.java)
- **Util**
  - Lesen einer Ressource auf der Festplatte (IOUtils.java)
- **View**
  - **Manager**
    - Laden und Verwalten von Shadern (ResourceManager.java)
  - **Renderer**
    - Interface für Renderer mit der Methode draw() (IPrimitiveRenderer.java)
    - Basisklasse für Renderer mit der Basisimplementierung von draw(). Beinhaltet Translation, Rotation, Skalierung und Farbsetzung für VertexArray (PrimitiveRenderer.java)
    - Renderer für geometrische Primitiven. Beinhaltet die geometrischen Daten (Circle, Square, Triangle).
    - TextRenderer.java: Lädt Font aus dem ressources Ordner, wandelt die Buchstaben in Bitmaps um und rendert diese in Quads anhand des übergebenen Textes.

- **Shader**
    - Shader.java: Kompiliert Shader und setzt diese zu einem Shader Programm zusammen.
    - Vertex und Fragment Shader für die geometrischen Primitiven und für Text.
- **OpenGLView.java**: Erstellen eines GLFW Fensters, Behandlung von Nutzereingaben, Initialisieren der Renderer und Rendern der Szene.

- **Client.java**: Startpunkt für einen Client. Kreiert NetworkManager, Game und OpenGLView.

Server

Das Server Package enthält Klassen und Packages, die nur für den Server relevant sind:

- **Controller**
    - **Game** (game.java)
        - Verwaltung der Datenstrukturen
        - Logik für Weltaktualisierungen, Kollisionsabfragen, Endbedingungen und Generierung der Daten, die an einen Client vermittelt werden.
    - **Network.Msg**
        - Handler die vom ClientManager, je nach einkommender Nachricht, aufgerufen werden und die geplante Logik ausführen (Move, Register, CheatCode, RequestStartGame).
    - **Network** (ClientManager.java)
        - Thread pro Client, der einkommenden Nachrichten annimmt und bearbeitet.
        - Sendet Nachrichten an den Client.
    - **Model.game.entity**
        - Datenstruktur der Entitäten die das Spiel besitzt (Food, Player, Ruin, Wall)
    - **Util**
        - Diamond Square Algorithmus zur dynamischen Generierung von Hindernissen (DiamondSquare.java)

        - QuadTree.java: Wird benutzt für die Kollisionsabfragen im Spiel und für die Generierung der Daten, die an einen Client vermittelt werden. Speichert für jeden Punkt im QuadTree die dazugehörigen Spieldaten und repräsentiert somit die Welt.
    - **Server.java**: Startpunkt für den Server. Erstellt einen ServerSocket, speichert Clients und verteilt Broadcast-Nachrichten an alle Clients,

Common

Das Common Package enthält Klassen und Packages, die für den Server und den Client relevant sind:

- **Game**
    - **Ai** (AIController.java)
        - Errechnet mit getNextMove() eine neue Richtung für einen AIClient
    - **Model**
        - Datenstrukturen die zwischen Client und Server geteilt werden (CheatCode, Direction, Food, Snake, etc.)
    - **Network**
        - Datenstrukturen für Nachrichten die zwischen Server und Client ausgetauscht werden (Move, GameUpdate, InitGame, etc.)
        - BaseMsg.java: Basisklasse für die Nachrichten. Erbt von Serializable.
        - MsgFactory.java: Erzeugt Nachrichten mit der Netzwerk ID des Senders.
    - **Configuration.java**: Liest die Konfigurationsdatei im resources Ordner ein und stellt die Properties zur Verfügung.

Im resources-Order befindet sich die Font OCRAEXT.TFF (für die Darstellung von Text), die Konfigurationsdatei config.properties (enthält Einstellungen für das Spiel), und die Konfigurationsdatei log4j2.xml für log4j.

Projektinstallation

Um das Programm lauffähig zu bekommen, müssen mit maven alle benötigten Dependencies geladen werden. Damit das Rendering von Text funktioniert, muss anschließend die Datei lib\freetype-jni\natives\freetype-jni-64.dll und lib\freetype-jni\freetpye64-jni_1.1.jar in das Projekt eingebunden werden. Voraussetzung dafür ist das Betriebssystem Windows in der 64-bit Version. Das Programm wurde mit Java 8 entwickelt. Um ein Spiel zu starten, muss zuerst ein Server gestartet werden und danach mindestens zwei Clients. Das Spiel startet sobald der steuerbare Client „Enter“ drückt. Mit WASD wird gesteuert. In der Konfigurationsdatei src\main\resources\config.properties können Spieleinstellungen geändert werden.

Es sind folgende Startskripte vorhanden:

- **RunAIClient.java**: startet einen AI Client
- **RunMultipleAIClients.java**: startet beliebig viele AI Clients
- **RunOpenGLClient.java**: startet einen steuerbaren Client mit UI
- **RunServer.java**: startet den Server

Netzwerk und Architektur

Das Projekt wurde mit der MVC-Architektur gebaut. Grundsätzlich gibt es drei Haupt-Packages: Common, Server und Client. Client und Server sind in Model, View und Controller unterteilt. Allerdings fehlt dem Client ein Model, da der Client nur die einkommenden Daten rendert. Dem Server hingegen fehlt eine View, da der Server nicht als Client in ein Spiel beitreten kann und deshalb keine View benötigt. Das Netzwerk im Spiel besteht aus beliebig vielen Clients und einem dezidierten Server. Der Server kann nicht gleichzeitig auch Spieler sein. Er berechnet die Logik des Spiels und sendet die Spielinformationen zu den Clients. Die Spielinformationen, die der Server versendet beinhalten alle Spielobjekte in dem Sichtradius des jeweiligen Client. Mehr über die Erstellung des Sichtradius wird im Abschnitt „Welt“ erläutert. Der Client nimmt diese Information an und rendert diese. Lediglich die Steuerungsbefehle, Cheatcodes und Spielstartanfragen werden vom Client an den Server geschickt. Für die Verbindung wird TCP benutzt, welches mit einem ObjectInputStream bzw. ObjectInputStream realisiert wird. Der Port vom ServerSocket kann in einer Konfigurationsdatei geändert werden.

Server

Der Server erstellt für jeden Client einen Thread – im Projekt ClientManager genannt. Der ClientManager Thread übernimmt die Kommunikation mit einem Client. Er sendet Nachrichten und nimmt Nachrichten entgegen. Werden Nachrichten entgegengenommen wird ein neuer Thread (MsgHandler) erzeugt. Dieser Thread führt die Schritte auf dem Server aus, die die Nachrichten fordern. Es gibt vier Typen einer Nachricht, die von einem Client kommen kann:

- Bewegung (MoveMsgHandler.java): enthält die Bewegung oben, unten, rechts oder links.
- Cheatcode (CheatCodeMsgHandler.java): enthält ein Enum, welches den Cheatcode repräsentiert (Cheatcodes wurden als Hilfe für das Debugging erstellt).
- Registrierung (RequestMsgHandler.java): ist eine leere Nachricht zur Client-Registrierung auf dem Server.
- Spielstartanfrage (RequestStartGameMsgHandler.java): ist eine leere Nachricht zur Anfrage zum Spielstart.

Die ClientManager, die den ObjectInputStream zum Client halten, identifizieren diese Nachrichten über den Objekttyp der Nachricht. Die Nachrichten werden mit einer Factory-Pattern-Methode (MsgFactory.java) erstellt. Diese fügt den Nachrichten automatisch die ID des Clients bzw. des Servers hinzu. So muss beim Erstellen einer Nachricht, nur die Daten für die Nachricht übergeben werden.

Client

Auf Seite des Clients wird ebenfalls ein Thread für die Kommunikation mit dem Server stellt – in diesem Projekt MsgReader genannt. Dieser wird vom NetworkManager erstellt und ist ebenfalls für das Senden und Empfangen von Nachrichten verantwortlich. Allerdings werden für einkommende Nachrichten keine Threads erstellt, weil der Client die Nachrichten in der Reihenfolge ausführen soll, in der die Nachrichten ankommen, und weil der Client nur von einer Entität Nachrichten bekommt. Auch hier werden die Nachrichten über den Objekttyp identifiziert. Der Client kann folgende Nachrichten empfangen:

- Spielstartinformation (InitGameMsg.java): leere Nachricht zur Information das das Spiel startet.
- Spielinformationen (GameUpateMsg.java): enthält die Spielinformationen im Sichtradius des Clients.
- Spielzustand (GameStateMsg.java): enthält ein Enum, welches den Spielzustand des Clients repräsentiert (Gewonnen oder Verloren)

Damit ein Netzwerkspiel zustande kommt, muss zuerst der Server gestartet werden. Die Clients, die mitspielen wollen, registrieren sich beim Server, wodurch ein ClientManager erstellt wird, welcher wiederum in eine Liste zur Verwaltung der Clients gespeichert wird. Ein Client kann jederzeit nach der Registrierung eine Spielstartanfrage schicken. Geschieht dies, wird ein neues Spiel erstellt. Das bedeutet eine neue Spielinstanz auf dem Server wird gestartet und den Spielern wird die Spielstartinformation gesendet. Diese Funktion kann mit der Taste „Enter“ vom Spieler ausgelöst werden und ist auch mitten im Spiel möglich.

Synchronisation

Der Codeabschnitt im MsgHandler für das Starten eines neuen Spiels wird mit einem Lock synchronisiert. Dies verhindert, dass mehrere Spieler gleichzeitig ein Spiel starten. Ebenso ist der Codeabschnitt im MoveMsgHandler mit einem Lock synchronisiert, damit

Bewegungen nicht zeitlich überschrieben werden können. An diesen Stellen konnte nicht mit dem Keyword synchronized gearbeitet werden, weil für jede Nachricht ein MsgHandler Thread erstellt wird. Und so läge das Synchronisationstoken in jedem Threadobjekt. Dieses soll jedoch global sein, damit mehrere Threads auf das gleiche Token darauf zugreifen können. Das keyword synchronized wurde zum Beispiel bei der Methode sendMessage im NetworkManger verwendet, damit beim häufigen Senden eines Cheatcodes nicht simultan auf den Output Stream zugegriffen wird. Da das Senden über den NetworkManger geschieht, der nur einmal existiert, kann das Keyword synchronized an dieser Stelle verwendet werden.

Limitationen

Die maximale Spieleranzahl liegt mit einem Surface Pro 7 (8GB) bei ungefähr 99 Spieler. Theoretisch sind mehr Spieler möglich, jedoch läuft der oben genannte Rechner unter normalen Entwicklungsbedingungen (ungefähr 6,2 GB belegter Speicher von 7,2 freier nutzbarer Speicher) in Hauptspeicher-Probleme, wenn sich mehr als 99 Spieler auf den Server verbinden. Bei der Erstellung der ClientManager Threads für die Clients auf dem Server, stürzt mit dem genannten Rechner, entweder das Betriebssystem oder die IDE ab, oder das Programm mit der Fehlermeldung:

 # A fatal error has been detected by the Java Runtime Environment: # # EXCEPTION_ACCESS_VIOLATION (0xc0000005) at pc=0x0000000070703ab0, pid=19768, tid=0x00000000000026c8

Die Spielgeschwindigkeit kann beliebig erhöht werden, jedoch stößt man bei hohen Spielerzahlen auf Grenzen. Eine sehr hohe Spielgeschwindigkeit bei hoher Spielerzahl, wird limitiert durch die Methode broadcastGameUpdateMsg() (*[Server] Server.java*). Diese iteriert über alle Spieler, errechnet die Spielinformationen für den jeweiligen Spieler und sendet die Spielinformationen über den jeweiligen ClientManager den Spielern - auch den Spielern, die schon tot sind, damit sie das Spiel nach dem Verlieren weiterverfolgen können. Die Methode ist synchronized, damit eine korrekte Reihenfolge der Aktualisierungen auf den Clients stattfinden kann. Die Threads, die das Spiel auf dem Server aktualisieren und diese Methode in einem Intervall aufrufen, sammeln sich bei hoher Aktualisierungsgeschwindigkeit bei dieser Methode, wodurch sich die Spielgeschwindigkeit nicht weiter verschnellert. Dies ist so, weil die Ausführung dieser Methode, je mehr Spieler

vorhanden sind, länger benötigt. Es kommt allerdings nicht zur Inkonsistenz der Spieldarstellung bei den Clients, da die wartenden Threads immer auf den gerade aktuellen Spielzustand zugreifen. Um dieses Problem vermeiden zu können, könnte man den Prozess der Informationsbeschaffung in einen neuen separaten Thread auslagern. Allerdings könnte man dadurch, wie bei der Erhöhung der Spieleranzahl, in Hauptspeicher-Probleme laufen.

Grafik

Zur Darstellung von Grafik wird OpenGL gewählt. Um mit Java auf OpenGL Funktionen zugreifen zu können, wird die Bibliothek LWJGL verwendet. OpenGL und das GLFW Fenster werden in der Klasse OpenGLView.java initialisiert. In dieser Klasse wird auch die Hauptschleife des Clients definiert. Diese besteht darin, Tastatureingaben abzufragen und die Spielinformationen, die asynchron über einen Thread empfangen werden, zu rendern. Vor dem Eintritt in diese Schleife werden jedoch erst die Renderer und die Shader erstellt.

Shader und Renderer

Die Erstellung der Shader findet mit der Klasse Shader.java statt. Mit dieser können Shader kompiliert und zu einem Shader Programm zusammengebaut werden. Shader werden mit der Klasse ResourceManager verwaltet. Diese Klasse bietet Funktionen, um Shader anhand einer Datei zu laden und zu einem späteren Zeitpunkt wieder zu bekommen. Es gibt zwei Shader:

- **Primitive**: für die Darstellung von primitiven geometrischen Formen
- **Text2D**: für die Darstellung von Text

Beide sind in einen Fragment und einen Vertex Shader unterteilt. Beide benutzen eine Projektionsmatrix, um Objekte abzuschneiden, die sich nicht auf dem Bildschirm befinden. Weil es sich um eine 2D Darstellung handelt, wird eine orthographische Projektion gewählt und die View-Matrix entfällt – könnte bzw. auch als eine Einheitsmatrix dargestellt werden - weil sich die Sicht auf den Raum nicht ändert. Für den Primitive Shader werden die Vertices im Vertex Shader noch zusätzlich mit der Model-Matrix multipliziert, damit die Primitiven in einem lokalen Raum definiert werden können, wodurch diese z.B. einfacherer skaliert werden können.

Die Shader werden in Renderern benutzt. Die Renderer der primitiven Formen leiten die Klasse PrimitiveRenderer ab und implementieren das Interface IPrimitiveRenderer. Die Klasse PrimitiveRenderer beinhaltet grundsätzliche Instruktionen, die jeder der primitiven Formen Renderer braucht. Dazu gehört das Einschalten des Shaders, die Erstellung der Model-Matrix und das Setzen der Model-Matrix und der Vertex Farbe in eine uniform Variable. IPrimitiveRenderer sichert, dass der Renderer die draw Methode implementiert,

die später in der Hauptschleife aufgerufen wird. Es existieren Renderer für Kreise, Rechtecke und Dreiecke.

Textrenderer

Der Textrenderer wurde mit dem Tutorial auf learnopengl.com erstellt. Aufgrund dessen, dass das Tutorial die C Bibliothek freetype (https://www.freetype.org/) verwendet, um Schriften zu laden und Bitmaps der Buchstaben zu erstellen, musste auf die Bibliothek freetype-jni zugegriffen werden. Diese Bibliothek ermöglicht es mit JNI mit Java auf die freetype Bibliothek zuzugreifen. Hierfür muss eine dll in das Projekt eingebunden werden. Die Generierung der Buchstaben als Bitmaps funktioniert einwandfrei, jedoch wird nichts auf dem Bildschirm angezeigt. Dies ist ein offener Bug. Allerdings hat die Anzeige von Text keine Priorität gehabt, weswegen die Textanzeige vorerst unvollständig im Code gelassen wurde. Die einzigen Stellen die Text gebraucht hätten, sind für die Nachrichten “Sie haben gewonnen” und “Sie haben verloren”. Als Ersatz wird an diesen Stellen ein grünes Quadrat in der Mitte des Fenster angezeigt, wenn man gewonnen hat, und ein rotes, wenn man verloren hat.

LWJGL

An manchen Stellen verschaffte die LWJGL Bibliothek Schwierigkeiten. Die Kenntnisse in C mit OpenGL kann man zwar weitestgehend anwenden, bei manchen Funktionsaufrufen werden allerdings andere Datenstrukturen erwartet. Zum Beispiel müssen Buffer von einer bestimmten Art sein (https://github.com/LWJGL/lwjgl3-wiki/wiki/1.3.-Memory-FAQ). Die Bibliothek liefert jedoch eine Klasse für das Erstellen solcher Buffer. Es ist jedoch, im Vergleich zu einem normalen C Programm, nicht so leicht verständlich, wie intern nun die Buffers aussehen und erzeugt werden. Außerdem muss für jede Methode, die man aus dem traditionellen OpenGL kennt, in den JavaDocs nachgeschaut werden, welches JWGL Pendant für die jeweilige Situation am besten geeignet ist. Methoden wie glVertexAttribPointer kommen in der Regel in mehreren Varianten: Eine moderne Variante, in Form von mehreren Überladenen Methoden, oder in der ursprünglichen nativen Variante, bei der man einen Pointer übergibt, die im Programmierkonzept von Java eigentlich nicht vorkommen.

Welt

Die Welt besteht aus einem Gitter und besteht aus 2^n + 1 Feldern. In der Welt existieren folgende Entitäten:

- Futter: Kann von einem Spieler eingesammelt werden, um die Spielerschlange zu vergrößern. Besetzt ein Feld.
- Spieler: Repräsentiert eine Spielerschlange, die mehrere Felder besetzen kann.
- Ruine: Eine Ruine darf von einem Spieler nicht berührt werden. Sie besetzt ein Feld.
- Mauer: Die Mauer umschließt das Spielfeld, wird mit der Zeit kleiner und darf nicht berührt werden.

Das Futter und die Spieler werden zum Spielstart zufällig auf das Spielfeld gesetzt. Bei den Spielern wird darauf geachtet, dass diese nicht zu nah aneinander liegen oder auf ein Futter gesetzt werden. Mit dem Fortschreiten der Zeit wird die Mauer kleiner. Zusätzlich zu dem Futter, wird bei Spielstart mit dem Diamond Square Algorithmus Ruinen generiert, die auf dem Spielfeld platziert werden. Der Diamond Square Algorithmus liefert einen 2D Array mit „schönen“ generierten Zufallszahlen. Die Zahlen sind auf einen Bereich zwischen 0 und 1 normiert. Alle Werte unter 0.2 sind Ruinen. Die Kollisionserkennung wird mit einem auf Punkte basierenden Quadtree durchgeführt.

Hauptschleife

Die Hauptschleife wird von einem separaten Thread in einem festen einstellbaren Intervall ausgeführt. Diese führt folgende Schritte nacheinander aus:

1. Bewege alle Spieler anhand der Benutzereingabe.
2. Führe „WorldEvent“ aus, welches nach einiger Zeit ausgelöst wird und den Spielrand verkleinert.
3. Überprüfe, ob ein Spieler mit Futter in Berührung kommt. Wenn ja, lasse die Schlange wachsen.
4. Überprüfe, ob ein Spieler in ein Objekt hineinläuft, das zum Tod führt (andere Spieler, Spielgrenze, eigene Schlange, Ruin). Wenn ja, dann sende dem Spieler die Nachricht, dass er verloren hat. Entferne Spieler von dem derzeitigen Spiel.

5. Überprüfe, ob ein Spieler gewonnen hat. Dies ist der Fall, wenn er der einzige Spieler auf dem Spielfeld ist. Wenn ja, dann sende dem Spieler die Nachricht, dass er gewonnen hat.
6. Sende allen Spielern die Spielinformationen, die in ihrem Sichtfeld vorhanden sind.

Sichtfeld

Ein Spieler sieht zu jeder Zeit nur ein Teil der Welt. Dieses Sichtfeld wird beim Einsammeln von Futter dynamisch vergrößert und stellt damit eine weitere Spielfunktion dar. Für die Berechnung dieses Sichtfeldes müssen die Koordinaten des globalen Gitters, welches vom Server verwaltet wird, in die lokalen Koordinaten des kleineren Spieler Gitters umgewandelt werden. Eine einfache Modulo Operation ist nicht geeignet, weil der Spieler immer in der Mitte seines Spielfeldes sein sollte. Außerdem muss beachtet werden, dass das Spielfeld nicht bei der Mauer, die die Spielgrenze darstellt, aufhört, sondern darüber hinaus geht. Der Spieler sieht also über die Mauer hinweg und die Welt ist quasi grenzenlos. Dieses Problem konnte einfach gelöst werden, indem die Welt von einem Quadtree repräsentiert wird (*generateGameDataQuadTree(), [Server] Game.java*), daraufhin die Punkte im Sichtradius des Spielers abgefragt werden (*getPlayerView(), [Server] Game.java*) und anhand des Sichtradius verschoben werden (*transformWorldSpaceToLocalSpace, [Server] Game.java*).

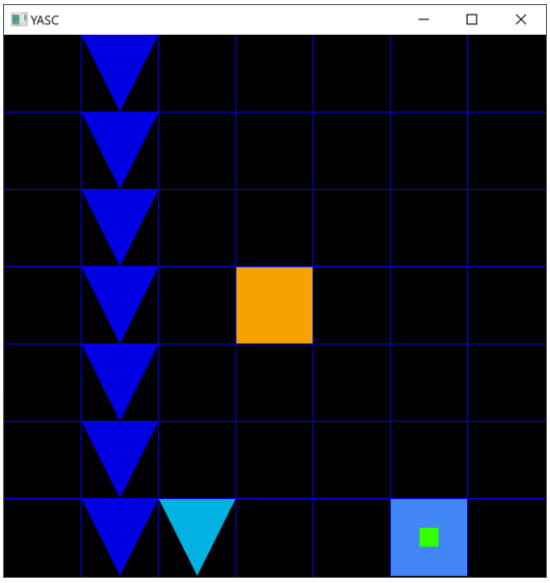

Abbildung 2: Lokale Ansicht (Sichtfeld)

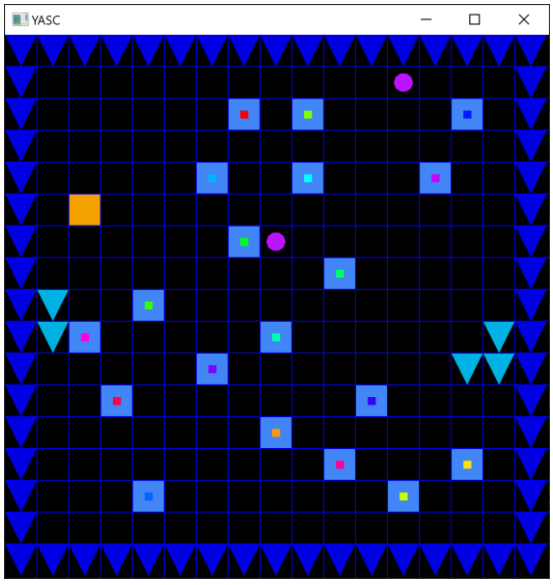

Abbildung 3: Globale Ansicht (abgeschnitten bei der Mauer)

Für jeden Spieler werden die aufgedeckten Ruinen gespeichert, um nachverfolgen zu können, welche Ruinen der Spieler schon aufgedeckt hat (siehe Abbildung 4 im Vergleich zu Abbildung 2 und 3).

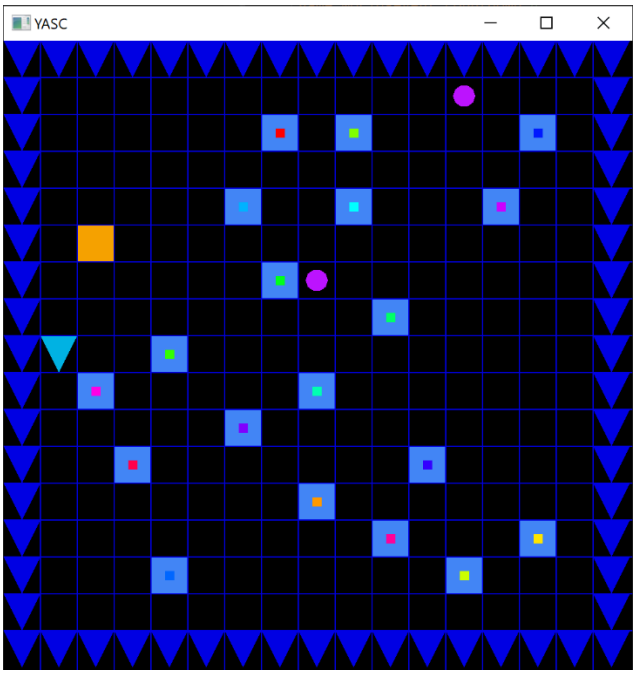

Abbildung 4: Die aufgedeckten Ruinen des Spielers im Vergleich zu Abbildung 2 und 3.

Externe Ressourcen

Für das Projekt wurden folgende externe Ressourcen verwendet:

QuadTree.java: Diese Java Klasse wurde von https://github.com/phishman3579/java-algorithms-implementation/blob/master/src/com/jweth erell/algorithms/ entnommen und leicht angepasst. Punkte im QuadTree speichern in der angepassten Variante Spielinformationen (z.B. Food oder Ruin).

DiamondSquare.java: Dieser Java Klasse wurde von https://stackoverflow.com/questions/2755750/diamond-square-algorithm entnommen und dient zur Generierung von Hindernissen (Ruins).

IOUtils.java: Diese Hilfsklasse stammt von dem LWJGL Demos Git Repository https://github.com/LWJGL/lwjgl3-demos/blob/main/src/org/lwjgl/demo/util/ und dient zum Einlesen von Dateien.

ResourceManager.java: ist ein selbstgeschriebener Java Port von der ursprünglichen C++ Variante von dem learnopengl.com Git Repository https://github.com/JoeyDeVries/LearnOpenGL/blob/master/src/7.in_practice/3.2d_game/0.full_source/resource_manager.cpp.

Shader.java: ist ein selbstgeschriebener Java Port von der ursprünglichen C++ Variante von dem learnopengl.com Git Repository https://github.com/JoeyDeVries/LearnOpenGL/blob/master/src/7.in_practice/3.2d_game/0.full_source/shader.cpp.

TextRenderer.java: ist ein selbstgeschriebener Java Port von dem learnopengl.com Tutorial https://learnopengl.com/In-Practice/Text-Rendering

Bibliotheken:

- **Log4J 2**: für das Logging im Programm (https://logging.apache.org/log4j/2.x/)
- **LWJGL**: für das Rendering des Spiels mit OpenGL (https://www.lwjgl.org/)
- **Freetype-jni**: für das Einlesen von Schriftarten und das Erstellen von Bitmaps von Buchstaben (https://github.com/mlomb/freetype-jni)

Build Tools:

- **Maven**: für das Dependency-Management (https://maven.apache.org/)